Learning Journal

Ian Silber

July 9, 2026

Contents

---

1 2026-06-25 *Catching up: What I have learned during my first month*

This month has been a fairly eventful one. I have spent most of my time working through Engineering Design via Surrogate Modelling A Practical Guide, as well as reading a few papers discussing the physics my research group is working to model.
What I have worked through thus far:

- Sampling plans, specifically Random Latin Hypercubes and criteria for determining which RLHC's are best.
- Understanding surrogate modelling
- Radial Basis Functions; both fixed and parametric RBFs
- Kriging Method for surrogate modelling, a special form of RBF.

1.1 Sampling Plans: Full Factorial, Random Latin Hypercubes, and Space Filling RLHCs

1.1.1 Full Factorial

```python
def full_factorial(q: list) -> None:

    if min(q) < 2:
        raise Exception("You must have at least two points per dimension")

    x1 = np.linspace(0,1,q[0])
    x2 = np.linspace(0,1,q[1])
    x3 = np.linspace(0,1,q[2])

    X1, X2, X3 = np.meshgrid(x1, x2, x3, indexing='ij')

    fig = plt.figure(figsize=(12, 12))

    elevations = [None, 90, 0, 0]
    azimuths   = [70, -90, -90, 0]

    for i, elev, azim in zip(range(1,5), elevations, azimuths):
        ax = fig.add_subplot(2,2,i, projection='3d')

        ax.xaxis.set_major_locator(MultipleLocator(1/(q[0]-1)))
        ax.xaxis.set_minor_locator(MultipleLocator(1/(q[0]-1)))

        ax.yaxis.set_major_locator(MultipleLocator(1/(q[1]-1)))
        ax.yaxis.set_minor_locator(MultipleLocator(1/(q[1]-1)))

        ax.zaxis.set_major_locator(MultipleLocator(1/(q[2]-1)))
        ax.zaxis.set_minor_locator(MultipleLocator(1/(q[2]-1)))

        ax.set_xlabel('x1')
        ax.set_ylabel('x2')
        ax.set_zlabel('x3')
        ax.scatter(X1, X2, X3, alpha=0.5)

        ax.view_init(elev=elev, azim=azim)
        ax.set_box_aspect(None, zoom=0.85)

```

```
    plt.show()

full_factorial([3,4,5])
```

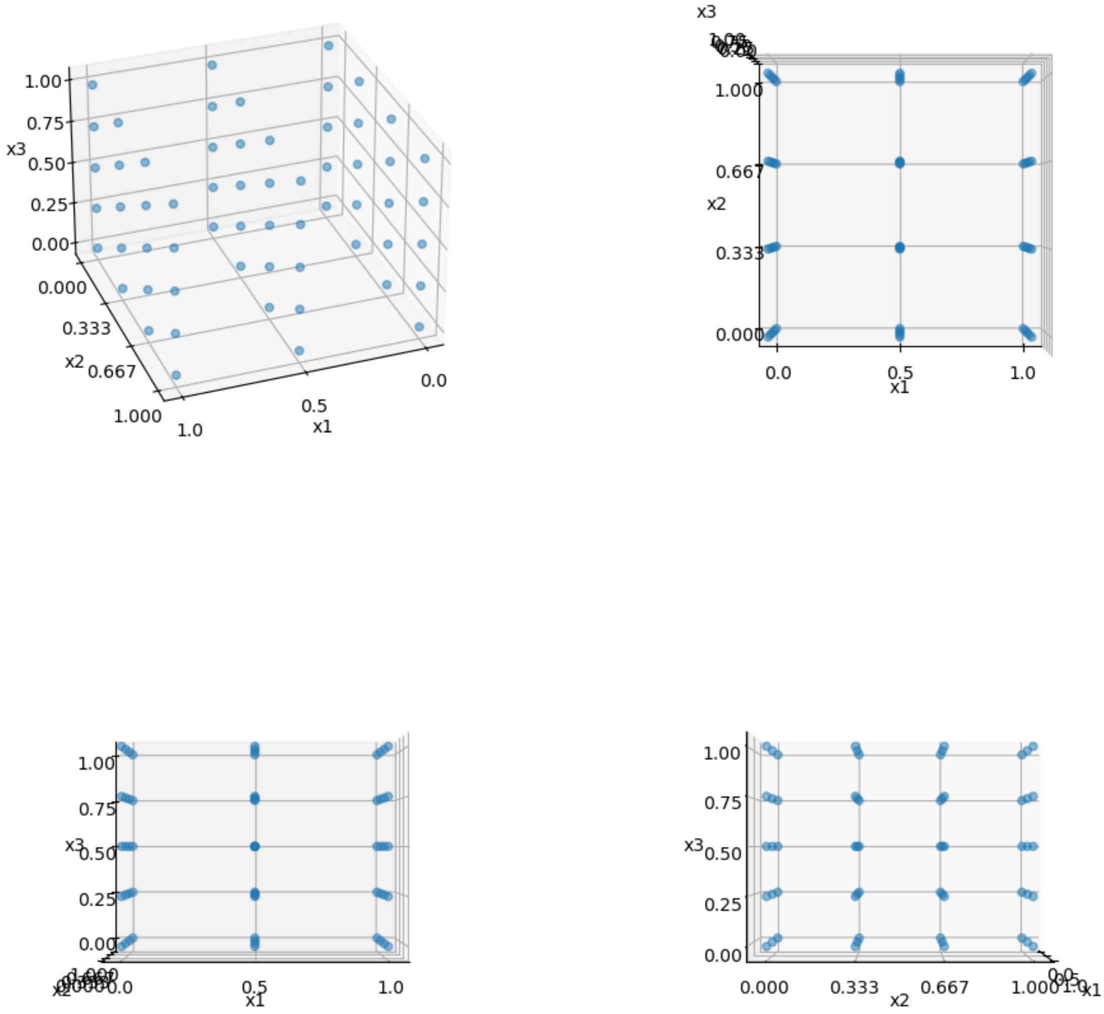

Figure 1: Output of the `space_filling_latin_hypercube` function viewed along different axis.

1.1.2 Random Latin Hypercubes

```
import numpy as np

def random_latin_hypercube(
        n:int,
        k:int,
        lbs: list = [],
        ubs: list = []
    ) -> np.ndarray:

    rng = np.random.default_rng()
```

```
    X = np.zeros((n,k))

    for i in range(k):
        X[:,i] = rng.permutation(n)

    for i, (ub, lb) in enumerate(zip(ubs, lbs)):
        X[:,i] = (X[:,i] + 0.5)/n
        w = ub - lb

        X_scaled = lb + w*X

    return X_scaled
```

1.1.3 Space Filling Random Latin Hypercubes

Although the above code ensures projection onto the axes does not result in multiple points overlapping, it does not result in maximal space-filling. Hypothetically, this code could produce a sampling plan in which all the points fall along a perfect diagonal through the feature space. This would avoid any overlapping points, but would be far from the most efficient sampling plan. This is where Johnson maximin metric and Morris and Mitchel's "tie-breaker" strategies come together to produce the space filling latin hypercube method.

Definition 1 (Maximin)**.** We call X a maximin plan among all plans if it maximizes d_1 and, among all plans for which this is true minimizes j_1 (where d_1 is distance between possible pairs of points sorted in ascending order, and j_1 is the number of pairs of points in X separated by d_1)

We want to maximize the smallest distance between pairs, and minimize the number of pairs that share this smallest distance.

This will sometimes result in a tie, so Morris and Mitchell's tie breaker method comes into play. Basically, its the same as Maximin but instead of stopping at maximizing d_1 and minimizing j_1, it maximizes $d_1 \dots d_m$ and minimizes $j_1 \dots j_m$.

$$d_p(x^{(i_1)}, x^{(i_2)}) = (\sum_{j=1}^{k} |x_j^{(i_1)} - x_j^{(i_2)}|)^{\frac{1}{p}} \tag{1}$$

The implementation of this plan is predictably a bit more involved:

```
import numpy as np
import math
from scipy.spatial.distance import pdist


def dist(X: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    # # My original approach:
    # distances = []
    # for i in range(len(X)-1):
    #     for j in range(1,len(X)):
    #         distances.append(np.linalg.norm(X[i,:] - X[j,:]))

    # # round because normalized to [0,1]
    # return np.unique(distances, return_counts=True) # returns
       distinct_d, J

    # claude suggested using pdists from scipy instead:
```

```python
    sq_dists = pdist(X, metric='sqeuclidean')
    return np.unique(sq_dists, return_counts=True) # returns distinct_d
        , J


def mm_phiq(X,q=2):
    d,J = dist(X)

    # morris mitchell sampling plan quality criterion
    return sum(((J/(d**q))**(1/q)))


def perturb(X: np.ndarray, pert_n=1):
    n = X.shape[0]
    k = X.shape[1]

    for _ in range(pert_n):
        col = int(np.floor(np.random.random()*k))

        el1 = el2 = 1
        while el1==el2:
            el1 = int(np.floor(np.random.random()*n))
            el2 = int(np.floor(np.random.random()*n))

        buffer = X[el1,col]
        X[el1,col] = X[el2,col]
        X[el2,col] = buffer

    return X


def best_lhc_by_q(X_start, pop, iter, q):
    X_best = X_start;
    phi_best = mm_phiq(X_start, q)

    n = X_best.shape[0]

    leveloff = math.floor(0.85*iter)
    for it in range(1,iter+1):
        if it < leveloff:
            mutations = round(1+(0.5*n-1)*(leveloff-it)/(leveloff-1))
        else:
            mutations = 1

        X_improved   = X_best
        phi_improved = phi_best

        for _ in range(pop):
            X_try = perturb(X_best, mutations)
            phi_try = mm_phiq(X_try, q)

            if phi_try < phi_improved:
                X_improved = X_try
                phi_improved = phi_try

        if phi_improved < phi_best:
            X_best = X_improved
            phi_best = phi_improved
```

```python

    return X_best, phi_best


def space_filling_latin_hypercube(
        n:int,
        k:int,
        lbs: list = [],
        ubs: list = [],
        pop:int = 25,
        iter:int = 25
    ) -> np.ndarray:
    """
    Generate a random Latin hypercube sampling plan in k dimensions.

    Each dimension is partitioned into n equal-width bins, and the plan
    places exactly one point per bin in every dimension (the Latin
    hypercube property), giving more uniform coverage of the domain than
    independent uniform sampling. Points are positioned at bin centres and
    then scaled from the unit hypercube [0, 1]^k onto [lb, ub]^k.

    Parameters
    ----------
    n : int
        Number of sample points. Also the number of bins per dimension,
        since the plan places one point per bin.
    k : int
        Number of dimensions (design variables).
    lbs : list, optional
        List of lower bound of the domain in every dimension (default 0).
    ubs : list, optional
        List of upper bound of the domain in every dimension (default 1).
        Must satisfy ub > lb.
    pop : int, optional
    iter : int, optional

    Returns
    -------
    np.ndarray
        An (n, k) array of sample points. Each column is a permutation of
        the same n bin centres, mapped to [lb, ub].

    Raises
    ------
    ValueError
        if ubs[i] <= lbs[i]

        if not len(ubs) == len(lbs) == k
    """
    X_start = random_latin_hypercube(n,k,lbs=lbs,ubs=ubs) # get sampling plan

    qs = [1,2,5,10,20,50,100]
```

```python

    X_tracker = {
        "q": "",
        "phi": math.inf,
        "X": ""
    }

    for q in qs:
        X_best, phi_best = best_lhc_by_q(X_start, pop, iter, q)

        if phi_best < X_tracker["phi"]:
            X_tracker["q"]   = q
            X_tracker["phi"] = phi_best
            X_tracker["X"]   = X_best

    return X_best


space_filling_latin_hypercube(25,3,25,25)
```

Which generates the below sampling plan:

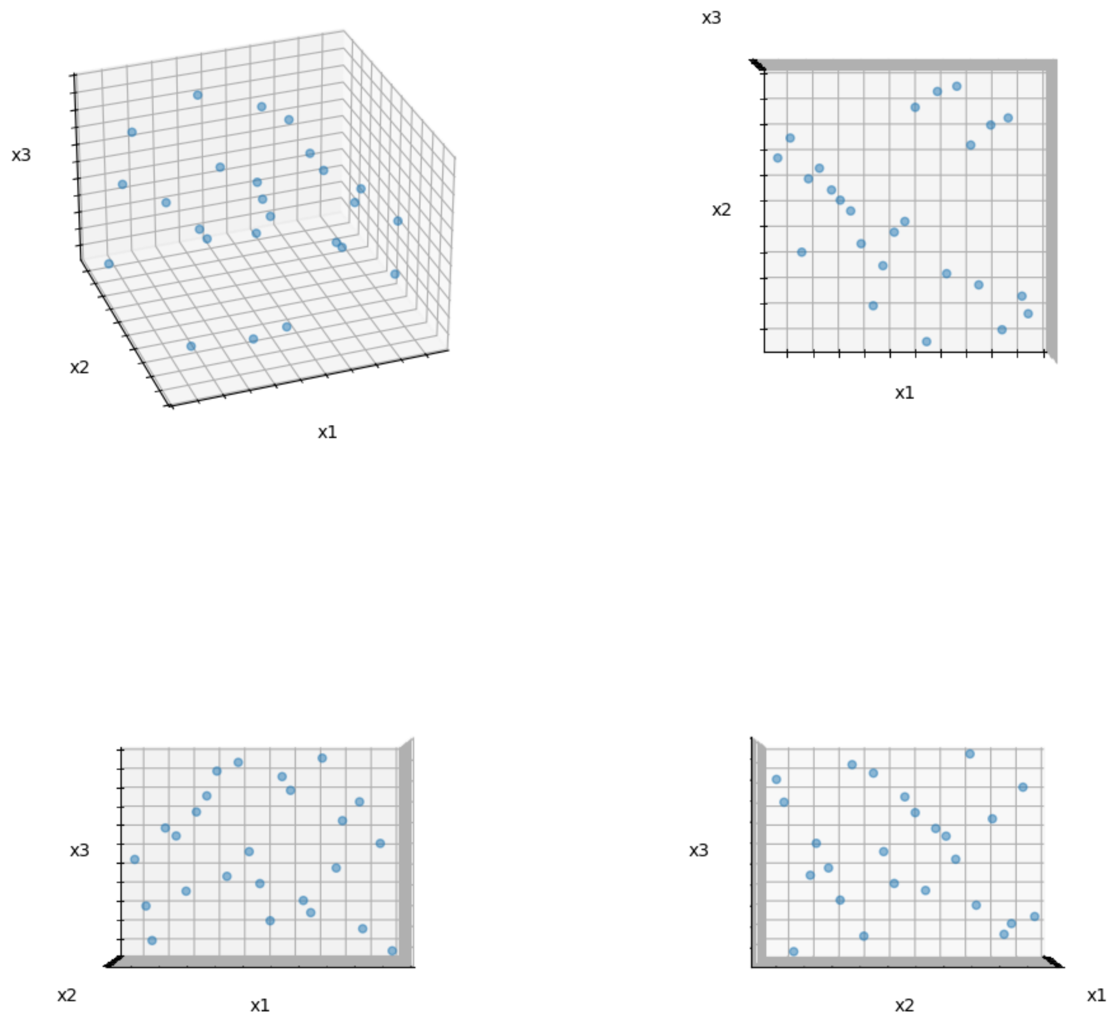

Figure 2: Output of the `space_filling_latin_hypercube` function viewed along different axis.

1.2 Radial Basis Functions: fixed and parametric

This should be below the figure

1.3 Kriging: A special form of RBF

Definition 2 (Symmetric Positive Definite (SPD) Matrix)**.** A real $n \times n$ matrix A is symmetric positive definite if it is symmetric ($A = A^T$) and

$$x^T Ax > 0 \quad \forall x.$$

A has a unique Cholesky factorization $A = R^T R$, where R is upper triangular with positive diagonal elements.
A has positive eigenvalues

Definition 3 (Maximum Likelihood Estimation (MLE))**.** We can either find the probability of the data given parameters, or find the parameters given the data. In the case of our Kriging models, we aim to estimate the parameters based on our limited data in order to create our surrogate.

Definition 4 (Stochastic Process)**.** A model of a system that evolves over time or space in a random unpredictable manner.

Definition 5 (Covariance)**.** A measure of the correlation between two or more sets of random variables

$$\begin{aligned} \operatorname{cov}(X,Y) &= E[(X-\mu_x)(Y-\mu_y)] && (2) \\ &= E[XY]-\mu_x\mu_y && (3) \end{aligned}$$

where μ_x and μ_y are the means of X and Y and E is the expectation. To make this make more sense, we can derive the more intuitive correlation from covariance by:

$$\operatorname{cor}(X,Y) = \frac{\operatorname{cov}(X,Y)}{\sigma_x\sigma_y} \tag{4}$$

(TODO) I still need to work on understanding what this means exactly.

2 2026-06-29 *Multi-Fidelity Analysis: Using multiple datasets for better approximations*

2.1 The general idea

In the event that we have multiple datasets describing a process, it is useful to combine data from an expensive function and a cheap approximation of said function. We want to sample the expensive function at a select few points that overlap with the sampling of the cheap function ($X_e \subset X_c$). Then we can try to correct points from Y_c with:

$$y_e = Z_p y_c + Z_d \tag{5}$$

2.2 Co-Kriging

From Forrester:

> Here we use the auto-regressive model of Kennedy and O'Hagan (2000), which assumes that
> $$\mathrm{cov}\{Y_e(x_i),\, Y_c(x) \mid Y_c(x_i)\} = 0 \quad \forall\, x \neq x_i.$$
> This means that no more can be learnt about $Y_e(x_i)$ from the cheaper code if the value of the expensive function at x_i is known (this is known as a *Markov property* which, in essence, says we assume that the expensive simulation is correct and any inaccuracies lie wholly in the cheaper simulation). (Forrester et al., 2008)

As in regular Kriging, we treat our expensive function as a stochastic process Z_e,

$$Z_e(\mathbf{x}) = \rho Z_c(\mathbf{x}) + Z_d(\mathbf{x}) \tag{6}$$

Basically, we are modeling the expensive function as our cheap function multiplied by some correction factor ρ plus a separate difference model Z_d that estimates the difference between our cheap and expensive function at a point.

It helps me to see the final product and then dissect the parts. To use our co-kriging model to estimate a point $x^{(n_e+1)}$ we evaluate:

$$\hat{y}_e\left(x^{(n_e+1)}\right) = \hat{\mu} + \mathbf{c}^T \mathbf{C}^{-1} \left(\mathbf{y} - \mathbf{1}\hat{\mu}\right) \tag{7}$$

Where:

$\hat{\mu}$ the estimated global mean of the co-Kriging process

$\mathbf{c}$ a column vector of correlations between the new point $x^{(n_e+1)}$ and the observed points

$$\mathbf{c} = \begin{pmatrix} \hat{\rho}\hat{\sigma}^2 \psi_c(\mathbf{X}_c, x^{(n+1)}) \\ \hat{\rho}^2\hat{\sigma}^2 \psi_c(\mathbf{X_e}, x^{(n+1)}) + \hat{\sigma}^2 \psi_d(\mathbf{X_e}, x^{(n+1)}) \end{pmatrix}$$

$\mathbf{C}$ the correlation matrix of the observed points

$$\mathbf{C} = \begin{pmatrix} \sigma^2 \mathbf{\Psi}_c(\mathbf{X}_c, \mathbf{X}_c) & \rho\sigma^2 \mathbf{\Psi}_c(\mathbf{X}_c, \mathbf{X}_e) \\ \rho\sigma_c^2 \mathbf{\Psi}_c(\mathbf{X}_e, \mathbf{X}_c) & \rho^2\sigma^2 \mathbf{\Psi}_c(\mathbf{X}_e, \mathbf{X}_e) + \sigma^2 \mathbf{\Psi}_d(\mathbf{X_e}, \mathbf{X}_e) \end{pmatrix}$$

$\mathbf{y}$ the vector of observed responses (cheap and expensive stacked)

$\mathbf{1}$ a column vector of ones, of the same length as $\mathbf{y}$

ψ or $\mathbf{\Psi}$ is a column vector or matrix calculated at each point with. Optimal theta values are found for each dimension and for both cheap points and expensive points (ψ_d)

$$\psi^{(i)} = \exp\left(-\sum_{j=1}^{k} \hat{\theta}_{j(\text{c or d})} \|x_j^{(n+1)} - x_j^{(i)}\|\right)$$

2.3 Co-Kriging – sampling plan revisited

From Forrester et al. (2007), our sampled points are:

$$\mathbf{X} = \begin{pmatrix} \mathbf{X_c} \\ \mathbf{X_e} \end{pmatrix} = (x_c^{(1)}, ..., x_c^{(n_c)}, x_e^{(1)}, ..., x_e^{(n_e)})^T \tag{8}$$

The difficulty is that

$$X_e \subset X_c$$

meaning we need to generate a random latin hypercube for our cheap points, and then pick the optimal subset of expensive points.

Thoughts. I think the easiest way to do this in code is to generate the optimal sampling plan for the subset, and then add points. The problem is that we need to be able to iterate over our sample set pairing one expensive sample point to one cheap sample point. I am trying to come up with an elegant solution that doesn't involve sorting the sample points in some messy way. Maybe I should just add points based on the Morris-Mitchell criterion one at a time? How can I pick a new point that maximizes distance from existing points? OK, I changed my mind, I think the optimal way to do this is finding a subset. If I try adding new points there is simply no way to ensure I have found the best plan. If I take a subset, I run into a similar problem. Since my expensive points must be a subset, I am not able to perturb the points, which means they will likely not be the best plan.

Here's what I came up with, I believe it is an accurate implementation of the exchange algorithm set out in Forrester et al. (2007) §4.

```

def find_subset(cube: np.ndarray, n: int, q: int = 2, iter: int = 1) ->
    np.ndarray:
    rng = np.random.default_rng()

    X_tracker = {
        "phi": math.inf,
        "X": np.empty([])
    }

    for _ in range(iter):
        # restart to avoid local minima
        X_best   = rng.choice(cube, size=n, replace=False)
        phi_best = mm_phiq(X_best, q)

        if phi_best < X_tracker["phi"]:
            X_tracker["phi"] = phi_best
            X_tracker["X"]   = X_best

        for i in range(n):
            # iterate over points in our subset
            X_test = X_tracker["X"].copy()

            for point in cube:
                # try each point from larger sample set
                X_test[i] = point
                phi_best  = mm_phiq(X_test, q)

                if phi_best < X_tracker["phi"]:
                    X_tracker["phi"] = phi_best
                    X_tracker["X"]   = X_test
```

```python

    return X_tracker["X"]
```

3 Papers

Contents

3.1 Guzman et al. (2026) — Toward Explaining GCR Modulation in a Dynamic Heliosheath

3.1.1 Definition Table

Heliosheath (HS) is the region of the heliosphere between the termination shock and the heliopause.

Spatial effects relate to persistent plasma properties affecting particle transport over large areas.

Temporal effects are transient modulation events and solar cycle variations advected outward from the Sun, generally lagging 1-2 yr from their creation to their arrival at the HS.

3.1.2 Abstract

GCR transport appears to be dominated by large, transient modulation structures advected from the Sun, while the role of spatially dictated diffusion is likely secondary.

Open question. What is spatially dictated diffusion?

3.2 Puzzoni et al. (2026) — A new web-based tool for GCR modulation in the heliosphere

3.3 M. C. Kennedy and A. O'Hagan (2000) — Predicting the output from a complex computer code

3.4 Forrester et al. (2007) — Multi-fidelity optimization via surrogate modelling